САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Домашняя работа № 2

Выполнил:

Гайдук Алина

БР 1.2

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

- Опишите все функциональные и нефункциональные требования к вашему бэкенд-приложению
- Выберите формат реализации API, которого вы будете придерживаться: REST API или Backend For Frontend
- Спроектируйте все эндпоинты вашего API в формате OpenAPI-схемы с учётом реализованной диаграммы БД
- Опишите тело каждого запроса и тело каждого ответа, продумайте возможные ошибки, возвращаемые из API

Ход работы

Для начала мне было необходимо расписать все функциональные и нефункциональные требования к приложению. Подумав над содержимым в ТЗ и просмотрев аналоги подобных приложений, я пришла к следующим функциональным требованиям:

1. Аутентификация и авторизация.
    - Нужно реализовать регистрацию нового пользователя (принимаем email, username, password, display_name).
    - Вход по почте и паролю с выдачей пары JWT токенов.
    - Обновление access токена по refresh токену.
    - Хеширование пароля.
2. Управление профилем.
    - Редактирование своего профиля через PATCH.
    - Просмотр профиля любого пользователя по id.
    - Просмотр рецептов и постов любого пользователя.
3. Рецепты.
    - Создание рецепта с шагами, ингредиентами, тегами, фото и видео (авторизованный пользователь).
    - Просмотр списка рецептов с фильтрацией по типу блюда, сложности, ингредиентам и тегам.
    - Поиск но названию рецепта.
    - Просмотр полного рецепта с шагами и ингредиентами, а также автором, лайками и комментариями.
    - Редактирование и удаление рецепта (только своего).
4. Посты.

    - Создание, просмотр, обновление и удаление постов.
    - Пагинация списка постов.
    - Обновление и удаление.
5. Комментарии.
    - Возможность комментирования рецептов и постов.
    - Поддержка вложенных ответов.
    - Получение дерева комментариев.
    - Удаление комментария.
6. Лайки.
    - Поставить или убрать лайк.
    - Возврат актуального счетчика лайков.
7. Подписки.
    - Подписка или отписка.
    - Получение списков подписчиков и подписок с пагинацией.
8. Сохраненные рецепты.
    - Сохранение или удаление рецепта из избранного.
    - Получение списка сохраненных рецептов.

Далее я перешла к нефункциональным требованиям. Здесь составить список было немного сложнее, но в итоге получился такой набор:

1. Валидация на уровне хендлера до обращения к БД.
2. Единый формат ошибок (JSON).
3. Пагинация.
4. Безопасность – хеширование паролей + использование access, refresh токенов в паре.
5. Логирование для отлова ошибок.
6. Конфигурация через переменные окружение, отсутствие захардкоженных значений в compose.

Определиться с форматом реализации для меня было достаточно просто, я выбрала REST API, мне бы хотелось попрактиковаться в написании бэкенда и по итогу курса получить хороший пет проект в портфолио.

Следующим шагом я перешла к проектированию эндпоинтов API. Для того, чтобы показать результат своей работы, я прикрепляю скриншоты

из Swagger. Полную OpenAPI схему можно посмотреть в файле openapi.yaml данного репозитория.

1. Auth

Auth Аутентификация и регистрация

- POST `/auth/register` Регистрация нового пользователя
- POST `/auth/login` Вход пользователя
- POST `/auth/refresh` Обновление access-токена

2. Users

Users Управление профилями пользователей

- GET `/users/{id}` Профиль пользователя
- PATCH `/users/me` Редактирование своего профиля
- GET `/users/{id}/recipes` Рецепты пользователя
- GET `/users/{id}/posts` Посты пользователя

3. Recipes

Recipes CRUD рецептов

- POST `/recipes` Создать рецепт
- GET `/recipes` Список рецептов с фильтрацией
- GET `/recipes/{id}` Полный рецепт
- PATCH `/recipes/{id}` Обновить рецепт (только автор)
- DELETE `/recipes/{id}` Удалить рецепт (только автор)

4. Posts

Posts CRUD постов (блог)

- POST `/posts` Создать пост
- GET `/posts` Список постов
- GET `/posts/{id}` Пост целиком
- PATCH `/posts/{id}` Обновить пост (только автор)
- DELETE `/posts/{id}` Удалить пост (только автор)

5. Comments

Comments Комментарии к рецептам и постам

POST	/recipes/{id}/comments	Комментарий к рецепту
GET	/recipes/{id}/comments	Комментарии к рецепту
POST	/posts/{id}/comments	Комментарий к посту
GET	/posts/{id}/comments	Комментарии к посту
DELETE	/comments/{id}	Удалить свой комментарий

6. Likes

Likes Лайки рецептов и постов

POST	/recipes/{id}/like	Поставить лайк рецепту
DELETE	/recipes/{id}/like	Убрать лайк с рецепта
POST	/posts/{id}/like	Поставить лайк посту
DELETE	/posts/{id}/like	Убрать лайк с поста

7. Follows

Follows Подписки на пользователей

POST	/users/{id}/follow	Подписаться на пользователя
DELETE	/users/{id}/follow	Отписаться от пользователя
GET	/users/{id}/followers	Подписчики пользователя
GET	/users/{id}/following	Подписки пользователя

8. Saved Recipes

Saved Recipes Сохранённые рецепты

POST	/recipes/{id}/save	Сохранить рецепт
DELETE	/recipes/{id}/save	Убрать рецепт из сохранённых
GET	/users/me/saved	Мои сохранённые рецепты

9. References (справочники)

References Справочники (типы блюд, сложность, теги, ингредиенты)

GET	/dish-types	Типы блюд
GET	/difficulties	Уровни сложности
GET	/tags	Список тегов
GET	/ingredients	Поиск ингредиентов (автокомплит)

Далее я описала формат запросов и ответов:

- Успешный ответ (один объект) - { "data": { ... } }
- Успешный ответ (список) – { "data": [...], "pagination": { "total": 100, "limit": 20, "offset": 0 } }
- Ответ без тела – HTTP 204 No Content — для операций удаления (DELETE).
- Все запросы и ответы используют application/json.

Все ошибки будут возвращаться в формате: {"error": {«code": "VALIDATION_ERROR", "message": "поле email обязательно", "details": [...] }}

Вывод

В ходе выполнения домашнего задания были описаны требования к сервису, спроектированы 30 эндпоинтов, охватывающих все сущности предметной области: аутентификация, пользователи, рецепты (с шагами, ингредиентами и тегами), посты, комментарии с поддержкой вложенности, лайки, подписки, сохранённые рецепты и справочники. Для каждого эндпоинта описаны тела запросов и ответов, учтены все возможные ошибки с единообразным форматом.